

UNIVERSITY OF SCIENCE
VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY



Lab Report

CS412 – Computer Vision

Individual Lab

Lab 01

22125085 – O Hon Sam

Theoretical Teacher : Tran Thai Son

Laboratory Teacher : Pham Minh Hoang
Vo Hoai Viet

Ho Chi Minh City, October 29, 2025

Contents

1	User Guide	2
2	Task 1: Image Loading	2
3	Task 2: RGB to Grayscale Conversion	2
4	Task 3: Brightness Adjustment	3
5	Task 4: Contrast Adjustment	4
6	Task 5: Average Filter and Separable Convolution	5
7	Task 6: Gaussian Filter and Separable Convolution	6
8	Task 7: Visualization	7

1 User Guide

The experiment was conducted on **Ubuntu 24.04** with OpenCV 4.10.0 and GCC 13. Compilation was managed using a Makefile.

To compile and execute the program, run the following commands:

```
$ make
$ ./bin/main -[flag] Lenna.png
```

The available command-line flags are:

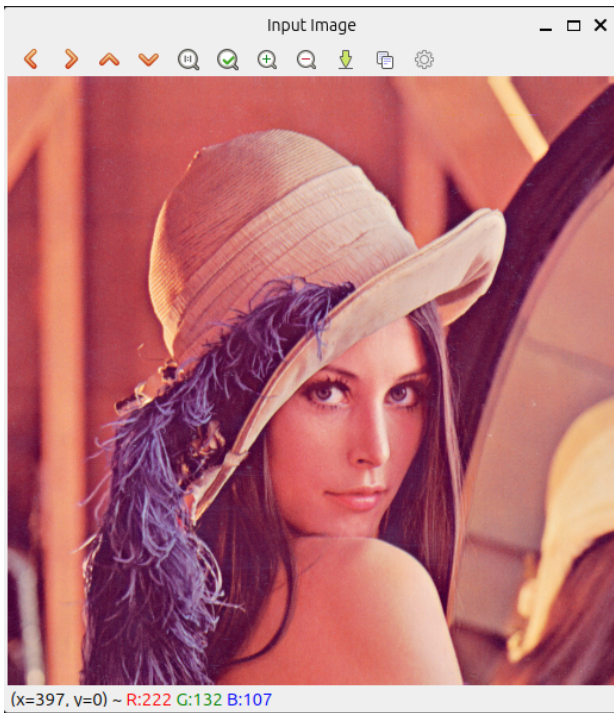
- **-rgb2gray**: Convert a color image to grayscale.
- **-brightness**: Adjust the brightness of the image.
- **-contrast**: Adjust the contrast of the image.
- **-avg**: Apply an average (mean) filter to the image.
- **-gauss**: Apply a Gaussian filter to the image.

2 Task 1: Image Loading

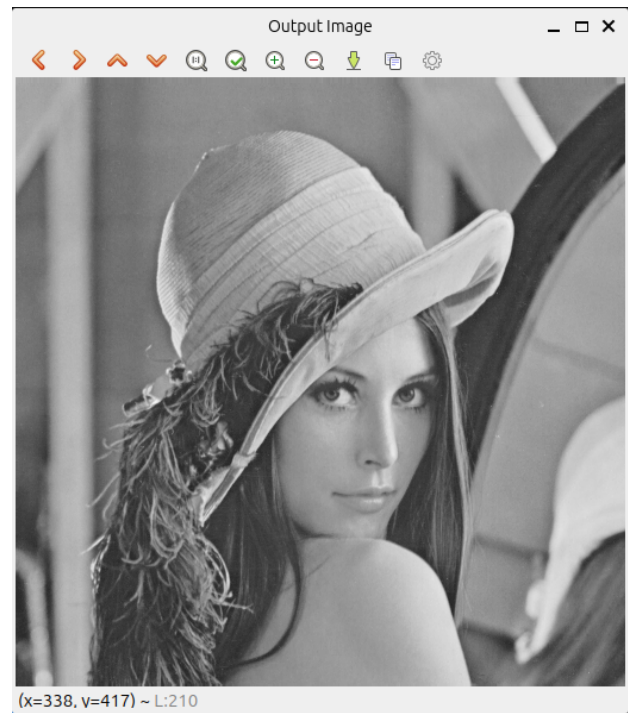
An image is loaded using the OpenCV `imread` function. It reads the image from a specified file path and stores it in a `cv::Mat` object, which represents a multidimensional dense numerical array (the core image structure in OpenCV).

3 Task 2: RGB to Grayscale Conversion

To convert a color image into grayscale, we pass a flag parameter to `imread` that specifies the read mode. When the command line includes **-rgb2gray**, the function uses the `IMREAD_GRAYSCALE` flag to convert the image internally into a single-channel grayscale representation.



(a) RGB image.



(b) Grayscale image.

Figure 1: Illustration of before and after applying `-rgb2gray` flag.

4 Task 3: Brightness Adjustment

To attach an interactive tracker to a display window, OpenCV provides the `createTrackbar` function¹. The adjustment of brightness and contrast is performed according to the linear transformation:

$$g(i, j) = \alpha \times f(i, j) + \beta, \quad (1)$$

where $\alpha > 0$ controls contrast and β controls brightness. The indices (i, j) denote the pixel position in the image. The formula is based on the OpenCV tutorial². Since we will access to each pixel separately, three nested loops are required to define row index, column index, and channel.

Instead of looping through each pixel manually, OpenCV also provides `cv::Mat::convertTo()`, which efficiently applies this transformation:

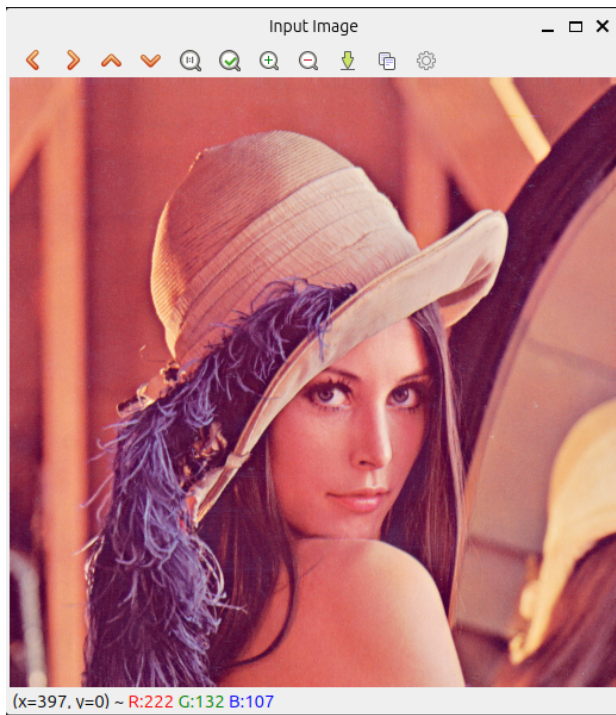
```
1 image.convertTo(new_image, -1, alpha, beta);
```

This operation computes:

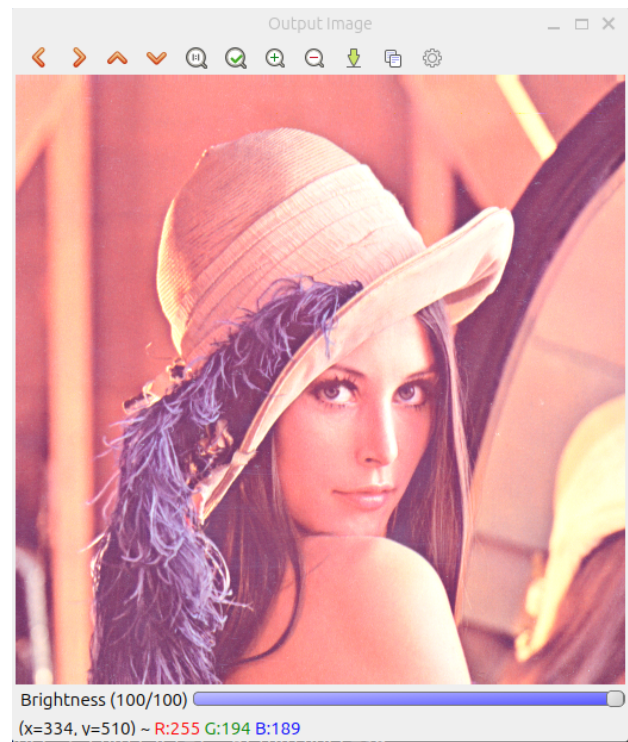
$$\text{new_image} = \alpha \cdot \text{image} + \beta.$$

¹<https://www.opencv-srf.com/2011/11/track-bars.html>

²[https://docs.opencv.org/3.4/d3/dc1/tutorial_basic_linear_transform.html#::~text=%7D-, Python, '\)](https://docs.opencv.org/3.4/d3/dc1/tutorial_basic_linear_transform.html#::~text=%7D-, Python, '))



(a) Original image.

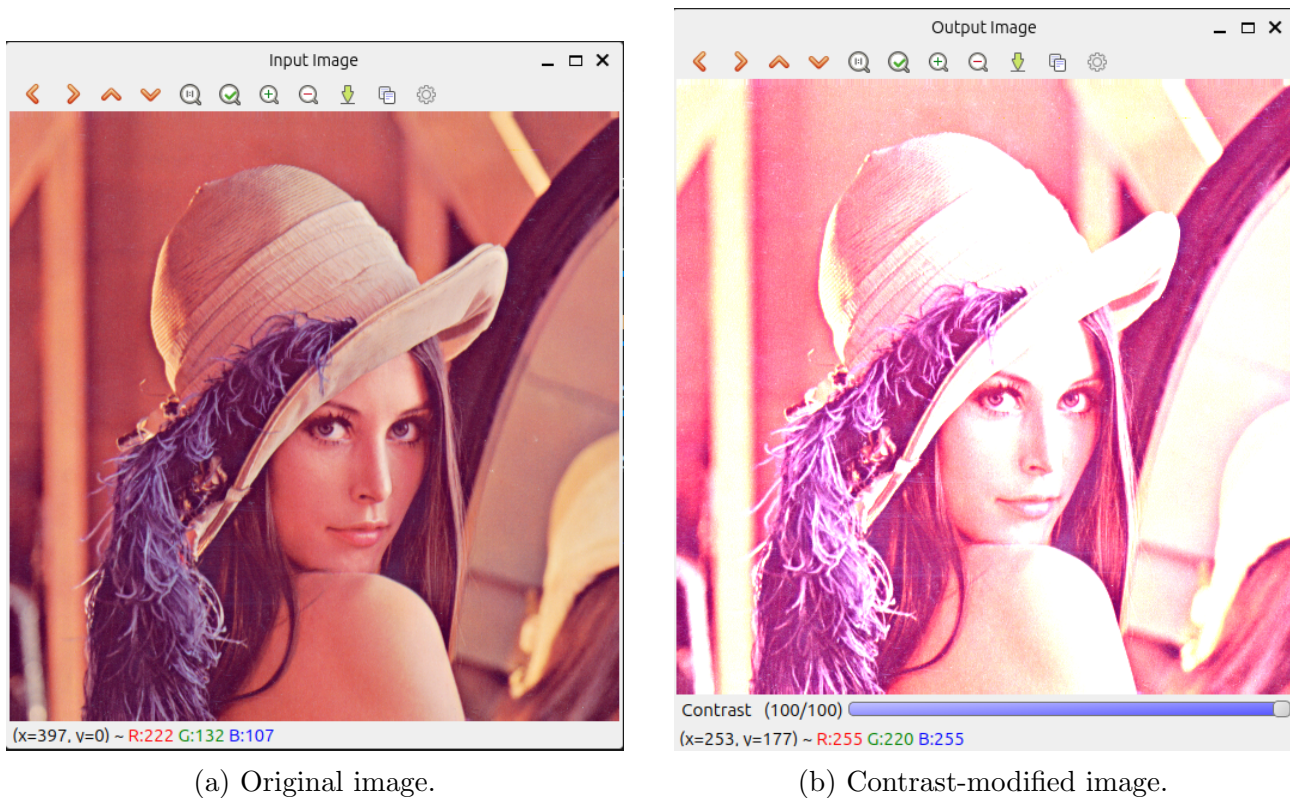


(b) Brightness-modified image.

Figure 2: Illustration of before and after applying `-brightness` flag.

5 Task 4: Contrast Adjustment

Task 4 builds upon Equation 1. Here, two trackbars are linked: one for brightness and one for contrast. The callback functions dynamically update the display by reapplying the linear transform with the user-controlled parameters α and β .



(a) Original image.

(b) Contrast-modified image.

Figure 3: Illustration of before and after applying `-contrast` flag.

6 Task 5: Average Filter and Separable Convolution

A **separable filter** in image processing can be expressed as the product of two simpler filters. A 2D convolution operation can thus be decomposed into two 1D convolutions, reducing the computational cost on an $N \times M$ image with an $n \times m$ filter from $\mathcal{O}(M \cdot N \cdot m \cdot n)$ to $\mathcal{O}(M \cdot N \cdot (m + n))^3$.

$$\text{Image} \otimes \text{Kernel}_{2D} = (\text{Image} \otimes \text{VerticalKernel}_{1D}) \otimes \text{HorizontalKernel}_{1D}. \quad (2)$$

For the average filter, we decompose a 2D kernel into two 1D kernels of size $k_{size} \times 1$ and $1 \times k_{size}$, each with element value $1/k_{size}$. The implementation uses zero-padding to ensure each image pixel aligns with the kernel center during convolution.

³https://en.wikipedia.org/wiki/Separable_filter

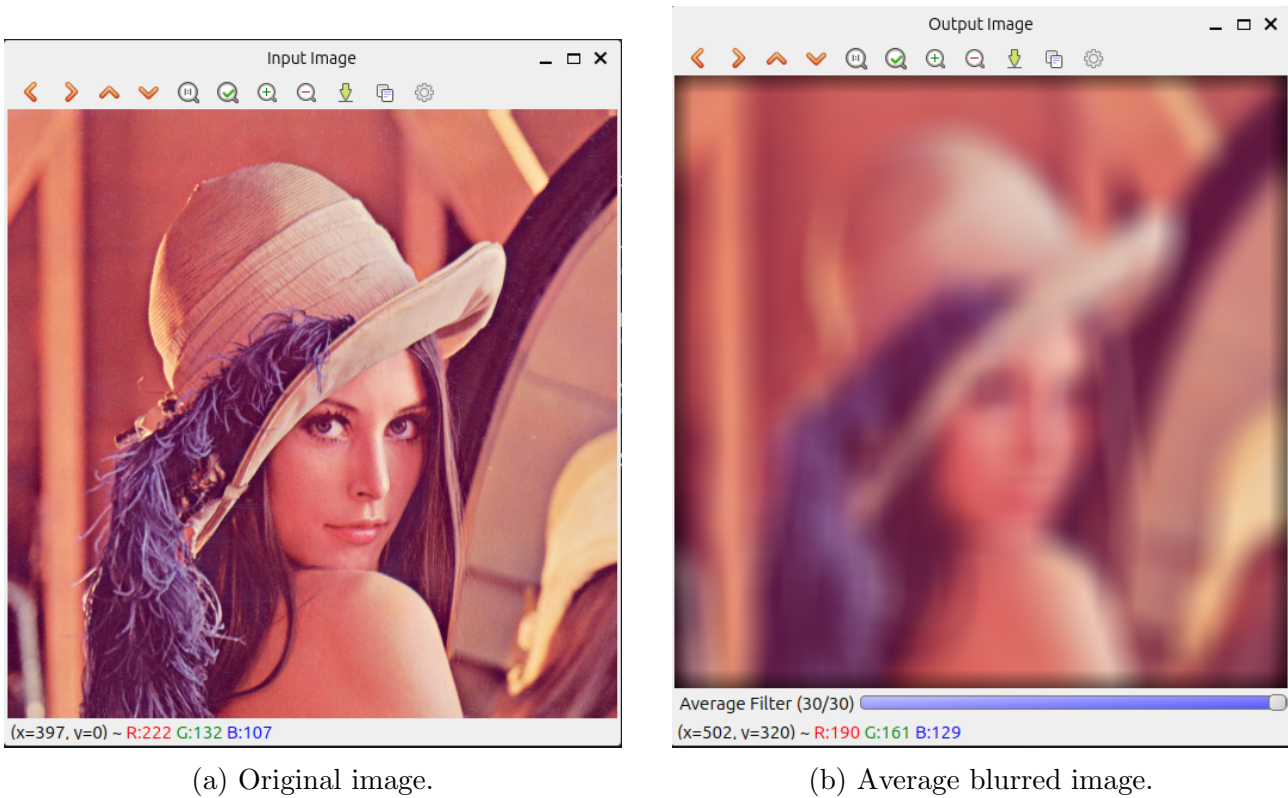


Figure 4: Illustration of before and after applying `-avg` flag.

7 Task 6: Gaussian Filter and Separable Convolution

Following the same separability principle, the 2D Gaussian filter is decomposed into two 1D Gaussian filters. The standard deviation σ is derived from the kernel size by:

$$\sigma = 0.3 \times \left(\frac{k_{size} - 1}{2} - 1 \right) + 0.8, \quad (3)$$

as described in the OpenCV documentation⁴.

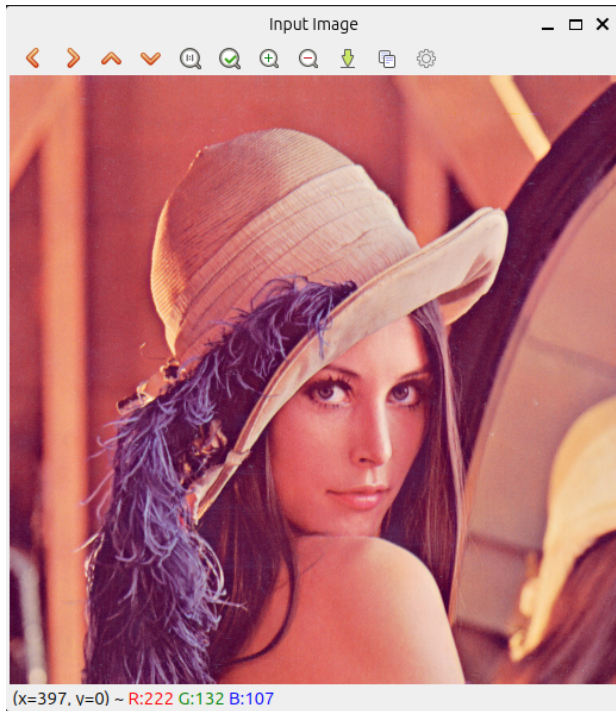
The 1D Gaussian filter coefficients are computed as:

$$G_i = \alpha \cdot e^{-\frac{(i - (k_{size} - 1)/2)^2}{2\sigma^2}}, \quad (4)$$

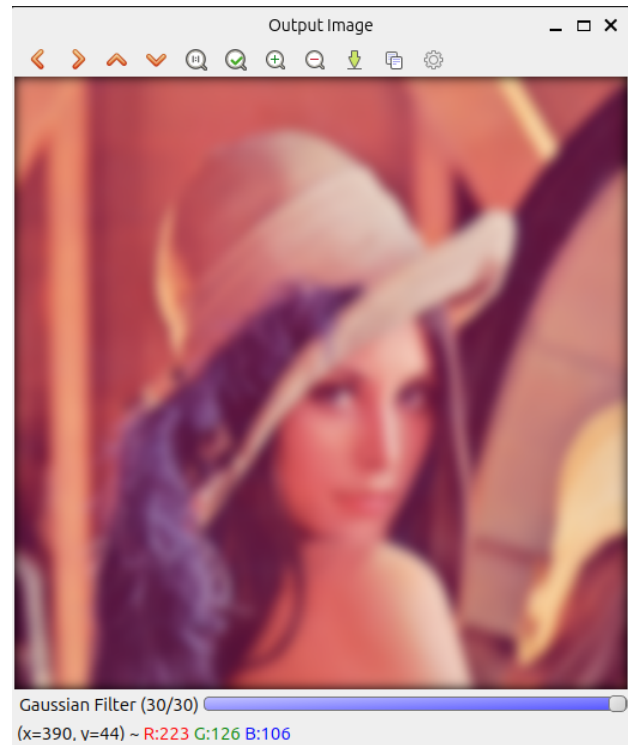
where $i = 0, \dots, k_{size} - 1$, and α is a normalization constant chosen so that $\sum_i G_i = 1$.

Geometrically, $(i - (k_{size} - 1)/2)$ represents the distance from the kernel center. The final Gaussian smoothing is achieved by applying the two 1D convolutions sequentially.

⁴https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html



(a) Original image.



(b) Gaussian blurred image.

Figure 5: Illustration of before and after applying `-gauss` flag.

8 Task 7: Visualization

The input and output images are displayed using OpenCV's `imshow` function. Two separate windows show the original and processed images in two separate windows.